

Tablas y tipos de datos

Prueba · Bases de datos relacionales · Nicolás Escudero

Parte I: películas y tags se relacionan N:N mediante película_tag. Parte II: respuestas vincula cada registro con un usuario y una pregunta mediante claves foráneas. Los archivos SQL definen los nombres y tipos del modelo.

prueba_fundamentos_nicolas_407/auditor@PostgreSQL loc...

No limit

Query Query HistoryScratch Pad

1-- Tablas y tipos de datos
2SELECT table_name,column_name,data_type,character_maximum_length
3FROM information_schema.columns
4WHERE table_schema='public'
5ORDER BY table_name,ordinal_position;

Data Output Messages Notifications

Showing rows: 1 to 17Page No: 1 of 1

	table_name name	column_name name	data_type character varying	character_maximum_length integer
1	pelicula_tag	pelicula_id	integer	[null]
2	pelicula_tag	tag_id	integer	[null]
3	peliculas	id	integer	[null]
4	peliculas	nombre	character varying	255
5	peliculas	anno	integer	[null]
6	preguntas	id	integer	[null]
7	preguntas	pregunta	character varying	255
8	preguntas	respuesta_correc...	character varying	[null]
9	respuestas	id	integer	[null]

Total rows: 17Query complete 00:00:00.092

Successfully run. Total query runtime: 92 msec. 17 rows affected.

CRLF Ln 5, Col 38

Claves y relación entre películas y tags

Prueba · Bases de datos relacionales · Nicolás Escudero

Cada entidad tiene clave primaria. pelicula_tag utiliza la clave compuesta (pelicula_id, tag_id), que impide repetir una asociación; sus claves foráneas mantienen la integridad referencial. respuestas enlaza usuarios y preguntas.

prueba_fundamentos_nicolas_407/auditor@PostgreSQL loc...

No limit

Query

Query History

Scratch Pad

```
-- Claves y relación entre películas y tags
SELECT conrelid::regclass AS tabla,conname AS restricción,pg_get_constraintdef(oid) AS definicion
FROM pg_constraint
WHERE connamespace='public'::regnamespace
ORDER BY tabla,restricción;
```

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 17Page No: 1 of 1

	tabla regclass	restricción name	definición text
1	películas	películas_id_not_null	NOT NULL id
2	películas	películas_pkey	PRIMARY KEY (id)
3	tags	tags_id_not_null	NOT NULL id
4	tags	tags_pkey	PRIMARY KEY (id)
5	película_t...	película_tag_película_id_fkey	FOREIGN KEY (película_id) REFERENCES películas(id)
6	película_t...	película_tag_película_id_not_...	NOT NULL película_id
7	película_t...	película_tag_pkey	PRIMARY KEY (película_id, tag_id)
8	película_t...	película_tag_tag_id_fkey	FOREIGN KEY (tag_id) REFERENCES tags(id)
9	película_t...	película_tag_tag_id_not_null	NOT NULL tag_id

Total rows: 17Query complete 00:00:00.058

✓ Successfully run. Total query runtime: 58 msec. 17 rows affected.

CRLF Ln 5, Col 28

Cinco películas y sus asociaciones

Prueba · Bases de datos relacionales · Nicolás Escudero

Se insertaron cinco películas y cinco tags. La película 1 se asocia a Drama, Aventura y Comedia; la película 2 a Familiar y Suspenso. Las restantes no tienen tags.

prueba_fundamentos_nicolas_407/auditor@PostgreSQL loc...

No limit

E

Query

Query History

Scratch Pad x

```
-- Cinco películas y sus asociaciones  
SELECT p.id,p.nombre,string_agg(t.tag,', '  
ORDER BY t.id) AS tags  
FROM peliculas p  
LEFT JOIN pelicula_tag pt ON pt.pelicula_id=p.id  
LEFT JOIN tags t ON t.id=pt.tag_id  
GROUP BY p.id  
ORDER BY p.id;
```

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 5

Page No:

1

of 1

	id [PK] integer	nombre character varying (255)	tags text
1	1	Película Uno	Drama, Aventura, Come...
2	2	Película Dos	Familiar, Suspenso
3	3	Película Tres	[null]
4	4	Película Cuatro	[null]
5	5	Película Cinco	[null]

Total rows: 5

Query complete 00:00:00.097

CRLF Ln 8, Col 15

Successfully run. Total query runtime: 97 msec. 5 rows affected.

Conteo de tags, incluidos ceros

Prueba · Bases de datos relacionales · Nicolás Escudero

LEFT JOIN conserva las películas sin asociaciones. COUNT(pt.tag_id) cuenta solo tags existentes y entrega 3, 2, 0, 0 y 0 para las cinco películas.

prueba_fundamentos_nicolas_407/auditor@PostgreSQL loc...

No limit

E

Query Query History

Scratch Pad x

```
1 -- Conteo de tags, incluidos ceros
2 SELECT p.id,p.nombre,count(pt.tag_id) AS cantidad_tags
3 FROM películas p
4 LEFT JOIN película_tag pt ON pt.película_id=p.id
5 GROUP BY p.id,p.nombre
6 ORDER BY p.id;
```

Data Output Messages Notifications

Showing rows: 1 to 5 Page No: 1 of 1

	id [PK] integer	nombre character varying (255)	cantidad_tags bigint
1	1	Película Uno	3
2	2	Película Dos	2
3	3	Película Tres	0
4	4	Película Cuatro	0
5	5	Película Cinco	0

Total rows: 5 Query complete 00:00:00.055

Successfully run. Total query runtime: 55 msec. 5 rows affected.

CRLF Ln 6, Col 15

Preguntas, usuarios y respuestas iniciales

Prueba · Bases de datos relacionales · Nicolás Escudero

Se crearon cinco preguntas, cinco usuarios y cinco respuestas. Dos usuarios responden correctamente la pregunta 1, uno acierta la pregunta 2 y las dos respuestas restantes son incorrectas. La comparación utiliza igualdad exacta del texto.

prueba_fundamentos_nicolas_407/auditor@PostgreSQL loc...

No limit

Query Query History Scratch Pad

```

1 -- Preguntas, usuarios y respuestas iniciales
2 SELECT r.id,u.nombre,p.pregunta,r.respuesta,p.respuesta_correcta,r.respuesta=p.respuesta_correcta AS
3 FROM respuestas r
4 JOIN usuarios u ON u.id=r.usuario_id
5 JOIN preguntas p ON p.id=r.pregunta_id
6 ORDER BY r.id;

```

Data Output Messages Notifications

SQL Showing rows: 1 to 5 Page No: 1 of 1

	id integer	nombre character varying (255)	pregunta character varying (255)	respuesta character varying (255)	respuesta_correcta character varying	correcta boolean
1	1	Ana	¿Cuánto es 2 + 2?	4	4	true
2	2	Bruno	¿Cuánto es 2 + 2?	4	4	true
3	3	Carla	¿Capital de Chile?	Santiago	Santiago	true
4	4	Diego	¿Lenguaje de este curs...	Java	Python	false
5	5	Elena	¿Cuánto es 3 + 3?	7	6	false

Total rows: 5 Query complete 00:00:00.049

✓ Successfully run. Total query runtime: 49 msec. 5 rows affected. ✗

CRLF Ln 6, Col 15

Respuestas correctas por usuario

Prueba · Bases de datos relacionales · Nicolás Escudero

El conteo agrupado por usuario aplica FILTER a las respuestas correctas y LEFT JOIN para conservar usuarios sin aciertos. Antes del borrado, los resultados son 1, 1, 1, 0 y 0.

[illegible]

Usuarios correctos por pregunta

Prueba · Bases de datos relacionales · Nicolás Escudero

COUNT(DISTINCT r.usuario_id) obtiene los usuarios que acertaron cada pregunta. La condición de respuesta correcta se incorpora al JOIN para conservar las preguntas con cero aciertos. Resultado: 2, 1, 0, 0 y 0.

prueba_fundamentos_nicolas_407/auditor@PostgreSQL loc...

No limit

QueryQuery HistoryScratch Pad x

1-- Usuarios correctos por pregunta
2SELECT p.id,p.pregunta,count(DISTINCT r.usuario_id) AS usuarios_correctos
3FROM preguntas p
4LEFT JOIN respuestas r ON r.pregunta_id=p.id AND r.respuesta=p.respuesta_correcta
5GROUP BY p.id,p.pregunta
6ORDER BY p.id;

Data OutputMessagesNotifications

SQL

Showing rows: 1 to 5Page No: 1of 1

	id [PK] integer	pregunta character varying (255)	usuarios_correctos bigint
1	1	¿Cuánto es 2 + 2?	2
2	2	¿Capital de Chile?	1
3	3	¿Lenguaje de este curso?	0
4	4	¿Cuánto es 3 + 3?	0
5	5	¿Motor de base de datos utiliza...	0

Total rows: 5Query complete 00:00:00.056

✓ Successfully run. Total query runtime: 56 msec. 5 rows affected.

CRLF Ln 6, Col 15

Eliminación del primer usuario y sus respuestas

Prueba · Bases de datos relacionales · Nicolás Escudero

La clave foránea de respuestas hacia usuarios se redefine con ON DELETE CASCADE. Al borrar el usuario 1 se elimina también su respuesta: quedan cuatro usuarios, cuatro respuestas y cero respuestas asociadas al usuario eliminado.

prueba_fundamentos_nicolas_407/auditor@PostgreSQL loc...

No limit

?

Query

Query History

Scratch Pad x

```
1  -- Eliminación del primer usuario y sus respuestas
2  ALTER TABLE respuestas DROP CONSTRAINT respuestas_usuario_id_fkey;
3  ALTER TABLE respuestas ADD CONSTRAINT respuestas_usuario_id_fkey
4  FOREIGN KEY(usuario_id) REFERENCES usuarios(id)
5  ON DELETE CASCADE;
6  DELETE
7  FROM usuarios
8  WHERE id=1;
9  SELECT (SELECT count(*)
10 FROM usuarios) AS usuarios_restantes, (SELECT count(*)
11 FROM respuestas) AS respuestas_restantes, (SELECT count(*)
12 FROM respuestas
13 WHERE usuario_id=1) AS respuestas_usuario_eliminado;
```

Data Output

Messages

Notifications

Showing rows: 1 to 1

Page No: 1 of 1

	usuarios_restantes bigint	respuestas_restantes bigint	respuestas_usuario_eliminado bigint
1	4	4	0

✓ Successfully run. Total query runtime: 45 msec. 1 rows affected. ✕

Total rows: 1 Query complete 00:00:00.045

CRLF Ln 13, Col 53

Edad mínima y email único

Prueba · Bases de datos relacionales · Nicolás Escudero

CHECK (edad >= 18) impide insertar edades menores de 18. ALTER TABLE añade email varchar(255) con restricción UNIQUE a la tabla existente. La clave primaria continúa identificando cada usuario; las restricciones quedan registradas en el catálogo de PostgreSQL.

prueba_fundamentos_nicolas_407/auditor@PostgreSQL loc...

No limit

Query Query HistoryScratch Pad x

1-- Edad mínima y email único
2ALTER TABLE usuarios ADD CONSTRAINT edad_adulta CHECK (edad>=18);
3ALTER TABLE usuarios ADD COLUMN email varchar(255) UNIQUE;
4SELECT conname AS restricción,pg_get_constraintdef(oid) AS definicion
5FROM pg_constraint
6WHERE conrelid='usuarios'::regclass
7ORDER BY conname;

Data Output Messages Notifications

Showing rows: 1 to 4Page No: 1 of 1

	restricción name	definición text
1	edad_adulta	CHECK ((edad >= 1...
2	usuarios_email_key	UNIQUE (email)
3	usuarios_id_not_n...	NOT NULL id
4	usuarios_pkey	PRIMARY KEY (id)

Total rows: 4Query complete 00:00:00.056

✓ Successfully run. Total query runtime: 56 msec. 4 rows affected.

CRLF Ln 7, Col 18